Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 9: Inheritance & Polymorphism (OOP-3)

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Class Relationships & Aggregation (Has-A)
2. Inheritance Basics (Is-A)
3. Constructor in Inheritance
4. Method Overriding
5. super() Keyword
6. Single & Multilevel Inheritance
7. Hierarchical, Multiple & Hybrid Inheritance
8. Method Overloading (Polymorphism)
9. Operator Overloading (Polymorphism)

54 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 9: Inheritance & Polymorphism (OOP-3) - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Class Relationships & Aggregation (Has-A), Inheritance Basics (Is-A), Constructor in Inheritance, Method Overriding, super() Keyword, Single & Multilevel Inheritance, Hierarchical, Multiple & Hybrid Inheritance, Method Overloading (Polymorphism), Operator Overloading (Polymorphism)
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Class Relationships & Aggregation (Has-A)

Q1. Customer Has an Address

BEGINNER

Create class Address(city, pin) and class Customer(name, address). A customer HAS-A address. Print the customer's city.

Solution:

```
class Address:
    def __init__(self, city, pin):
        self.city = city
        self.pin = pin

class Customer:
    def __init__(self, name, address):
        self.name = name
        self.address = address

addr = Address("Pune", 411018)
c = Customer("Riya", addr)
print(c.name, c.address.city)
```

Output:

```
Riya Pune
```

Q2. Car Has an Engine

BEGINNER

Create class Engine(hp) and class Car(model, engine). Print the car model and engine horsepower.

Solution:

```
class Engine:
    def __init__(self, hp):
        self.hp = hp

class Car:
    def __init__(self, model, engine):
        self.model = model
        self.engine = engine

car = Car("Nexon", Engine(120))
print(car.model, car.engine.hp)
```

Output:

```
Nexon 120
```

Q3. Shared Object in Aggregation

INTERMEDIATE

Two employees share the same Office object. Change the office floor once and print it for both employees.

Solution:

```
class Office:
    def __init__(self, floor):
        self.floor = floor

class Employee:
    def __init__(self, name, office):
        self.name = name
        self.office = office

off = Office(3)
e1 = Employee("Om", off)
e2 = Employee("Pia", off)
off.floor = 5
print(e1.office.floor, e2.office.floor)
```

Output:

```
5 5
```

Q4. Object Lives Independently

INTERMEDIATE

Delete a Department object that contained a Professor. Show the Professor object still exists (aggregation, not ownership).

Solution:

```
class Professor:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, prof):
        self.prof = prof

p = Professor("Dr. Rao")
d = Department(p)
del d
print(p.name)
```

Output:

```
Dr. Rao
```

Q5. Order Has Many Items

ADVANCED

Create class Item(name, price, qty) and class Order with a list of items, add_item() and total(). Add 3 items and print the total.

Solution:

```
class Item:
    def __init__(self, name, price, qty):
        self.name = name
        self.price = price
        self.qty = qty

class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def total(self):
        return sum(i.price * i.qty for i in self.items)

o = Order()
o.add_item(Item("Pen", 10, 5))
o.add_item(Item("Book", 60, 2))
o.add_item(Item("Bag", 450, 1))
print(o.total())
```

Output:

```
620
```

Q6. Accessing Private Data Through Getter

ADVANCED

Address has a private __pin. Customer prints the pin through address.get_pin() since it cannot read __pin directly.

Solution:

```
class Address:
    def __init__(self, city, pin):
        self.city = city
        self.__pin = pin

    def get_pin(self):
        return self.__pin

class Customer:
    def __init__(self, name, address):
        self.name = name
        self.address = address

    def show(self):
        pin = self.address.get_pin()
        print(self.name, self.address.city, pin)

Customer("Om", Address("Nashik", 422001)).show()
```

Output:

```
Om Nashik 422001
```

2. Inheritance Basics (Is-A)

Q7. Child Uses Parent Method

BEGINNER

Create class Vehicle with method start(). Create class Bike(Vehicle) with no code. Call start() on a Bike object.

Solution:

```
class Vehicle:
    def start(self):
        print("Vehicle started")

class Bike(Vehicle):
    pass

b = Bike()
b.start()
```

Output:

```
Vehicle started
```

Q8. Child Adds Its Own Method

BEGINNER

Class Animal has eat(). Class Dog(Animal) adds bark(). Call both on a Dog object.

Solution:

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Woof")

d = Dog()
d.eat()
d.bark()
```

Output:

```
Eating
Woof
```

Q9. isinstance and issubclass

INTERMEDIATE

For classes Employee and Manager(Employee), check isinstance(m, Employee) and issubclass(Manager, Employee).

Solution:

```
class Employee:
    pass

class Manager(Employee):
    pass

m = Manager()
print(isinstance(m, Manager))
print(isinstance(m, Employee))
print(issubclass(Manager, Employee))
print(issubclass(Employee, Manager))
```

Output:

```
True
True
True
False
```

Q10. What is Inherited?

INTERMEDIATE

Parent class has a class variable company, a public attribute and a private attribute. Show which ones a child object can access.

Solution:

```
class Parent:
    company = "Bikkad IT"

    def __init__(self):
        self.city = "Pune"
        self.__secret = "hidden"

class Child(Parent):
    pass

c = Child()
print(c.company)
print(c.city)
try:
    print(c.__secret)
except AttributeError as e:
    print("Private not accessible")
```

Output:

```
Bikkad IT
Pune
Private not accessible
```

Q11. Parent Cannot Use Child Method

ADVANCED

Show that a Vehicle object cannot call Car's method honk() (inheritance works only downward).

Solution:

```
class Vehicle:
    pass

class Car(Vehicle):
    def honk(self):
        print("Beep beep")

Car().honk()
try:
    Vehicle().honk()
except AttributeError as e:
    print(e)
```

Output:

```
Beep beep
'Vehicle' object has no attribute 'honk'
```

Q12. Method Resolution Order

ADVANCED

Print Car.__mro__ names for class Car(Vehicle) to see the order Python searches for methods.

Solution:

```
class Vehicle:
    pass

class Car(Vehicle):
    pass

for cls in Car.__mro__:
    print(cls.__name__)
```

Output:

```
Car
Vehicle
object
```

3. Constructor in Inheritance

Q13. Child Without Constructor

BEGINNER

Parent Person has a constructor that sets name. Child Student has no constructor. Create Student("Om") and print name.

Solution:

```python
class Person:
    def __init__(self, name):
        self.name = name
        print("Person constructor")

class Student(Person):
    pass

s = Student("Om")
print(s.name)
```

Output:

```
Person constructor
Om
```

Q14. Child With Its Own Constructor

BEGINNER

If Student has its own constructor that sets roll only, show that the parent constructor is NOT called and name is missing.

Solution:

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, roll):
        self.roll = roll

s = Student(21)
print(s.roll)
print(hasattr(s, "name"))
```

Output:

```
21
False
```

Q15. Call Parent Constructor with super()

INTERMEDIATE

Fix the Student class so its constructor calls super().__init__(name) and also sets roll.

Solution:

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, roll):
        super().__init__(name)
        self.roll = roll

s = Student("Om", 21)
print(s.name, s.roll)
```

Output:

```
Om 21
```

Q16. Getter for Parent Private Data

INTERMEDIATE

Parent Account has private __balance and get_balance(). Child SavingsAccount uses get_balance() to print the balance.

Solution:

```python
class Account:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

class SavingsAccount(Account):
    def show(self):
        print(self.get_balance())

SavingsAccount(7000).show()
```

Output:

```
7000
```

Q17. Order of Constructors

ADVANCED

Create a 3-level chain A -> B -> C, each constructor printing its name and calling super(). Create a C object.

Solution:

```
class A:
    def __init__(self):
        print("A init")

class B(A):
    def __init__(self):
        super().__init__()
        print("B init")

class C(B):
    def __init__(self):
        super().__init__()
        print("C init")

C()
```

Output:

```
A init
B init
C init
```

Q18. Employee and Manager

ADVANCED

Employee(name, salary). Manager(name, salary, team_size) adds bonus = 1000 per team member. Print the manager's total pay.

Solution:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

class Manager(Employee):
    def __init__(self, name, salary, team_size):
        super().__init__(name, salary)
        self.team_size = team_size

    def total_pay(self):
        return self.salary + self.team_size * 1000

m = Manager("Seema", 80000, 6)
print(m.name, m.total_pay())
```

Output:

```
Seema 86000
```

4. Method Overriding

Q19. Override a Message

BEGINNER

Parent Notification has send() printing "Sending notification". Child SMS overrides it to print "Sending SMS".

Solution:

```
class Notification:
    def send(self):
        print("Sending notification")

class SMS(Notification):
    def send(self):
        print("Sending SMS")

Notification().send()
SMS().send()
```

Output:

```
Sending notification
Sending SMS
```

Q20. Different Sounds

BEGINNER

Animal.sound() returns "...". Override it in Cat and Cow. Loop through a list of animals and print each sound.

Solution:

```
class Animal:
    def sound(self):
        return "..."

class Cat(Animal):
    def sound(self):
        return "Meow"

class Cow(Animal):
    def sound(self):
        return "Moo"

for a in [Animal(), Cat(), Cow()]:
    print(a.sound())
```

Output:

```
...
Meow
Moo
```

Q21. Area of Shapes

INTERMEDIATE

Shape.area() returns 0. Override it in Rectangle(l, w) and Circle(r). Print the areas.

Solution:

```python
import math

class Shape:
    def area(self):
        return 0

class Rectangle(Shape):
    def __init__(self, l, w):
        self.l = l
        self.w = w

    def area(self):
        return self.l * self.w

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return round(math.pi * self.r ** 2, 2)

for s in [Rectangle(4, 5), Circle(3)]:
    print(s.area())
```

Output:

```
20
28.27
```

Q22. Override __str__

INTERMEDIATE

Override __str__ in class Product so print(product) shows "Laptop: Rs.55000".

Solution:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name}: Rs.{self.price}"

print(Product("Laptop", 55000))
```

Output:

```
Laptop: Rs.55000
```

Q23. Delivery Charges by Type

ADVANCED

Delivery.charge(km) = 10 per km. Express overrides it to 15 per km + 50 fixed. Print charges for 8 km for both.

Solution:

```
class Delivery:
    def charge(self, km):
        return km * 10

class Express(Delivery):
    def charge(self, km):
        return km * 15 + 50

print(Delivery().charge(8))
print(Express().charge(8))
```

Output:

```
80
170
```

Q24. Polymorphic Payment Processing

ADVANCED

Payment.pay(amount) is overridden in UPI, Card (2% fee) and Cash. Process a list of payments for 1000 each.

Solution:

```
class Payment:
    def pay(self, amount):
        return amount

class UPI(Payment):
    def pay(self, amount):
        return f"UPI paid {amount}"

class Card(Payment):
    def pay(self, amount):
        return f"Card paid {amount + amount * 0.02}"

class Cash(Payment):
    def pay(self, amount):
        return f"Cash paid {amount}"

for p in [UPI(), Card(), Cash()]:
    print(p.pay(1000))
```

Output:

```
UPI paid 1000
Card paid 1020.0
Cash paid 1000
```

5. super() Keyword

Q25. Extend a Parent Method

BEGINNER

Parent Report.header() prints "Bikkad IT". Child SalesReport.header() calls super().header() and then prints "Sales Report".

Solution:

```
class Report:
    def header(self):
        print("Bikkad IT")

class SalesReport(Report):
    def header(self):
        super().header()
        print("Sales Report")

SalesReport().header()
```

Output:

```
Bikkad IT
Sales Report
```

Q26. super() in Constructor

BEGINNER

Parent Vehicle(wheels). Child Bike calls super().__init__(2) and adds brand.

Solution:

```
class Vehicle:
    def __init__(self, wheels):
        self.wheels = wheels

class Bike(Vehicle):
    def __init__(self, brand):
        super().__init__(2)
        self.brand = brand

b = Bike("Hero")
print(b.brand, b.wheels)
```

Output:

```
Hero 2
```

Q27. Add Extra Calculation

INTERMEDIATE

Parent Bill.total() returns amount. Child GSTBill.total() uses super().total() and adds 18% GST.

Solution:

```
class Bill:
    def __init__(self, amount):
        self.amount = amount

    def total(self):
        return self.amount

class GSTBill(Bill):
    def total(self):
        base = super().total()
        return base + base * 18 / 100

print(Bill(1000).total())
print(GSTBill(1000).total())
```

Output:

```
1000
1180.0
```

Q28. super() Outside a Class

INTERMEDIATE

Try to use super() outside any class inside try-except and print the error.

Solution:

```
try:
    super().__init__()
except RuntimeError as e:
    print(e)
```

Output:

```
super(): no arguments
```

Q29. Chained super() Calls

ADVANCED

A Logger chain: BaseLogger.log() prints "[LOG]", TimeLogger adds "[TIME]", UserLogger adds "[USER]". Each calls super().log() first.

Solution:

```
class BaseLogger:
    def log(self, msg):
        print("[LOG]", msg)

class TimeLogger(BaseLogger):
    def log(self, msg):
        super().log(msg)
        print("[TIME] 10:30")

class UserLogger(TimeLogger):
    def log(self, msg):
        super().log(msg)
        print("[USER] admin")

UserLogger().log("Server started")
```

Output:

```
[LOG] Server started
[TIME] 10:30
[USER] admin
```

Q30. super() with Multiple Inheritance (MRO)

ADVANCED

Classes A, B(A), C(A), D(B, C) all have hello() calling super().hello(). Call D().hello() and print D's MRO.

Solution:

```
class A:
    def hello(self):
        print("A")

class B(A):
    def hello(self):
        print("B")
        super().hello()

class C(A):
    def hello(self):
        print("C")
        super().hello()

class D(B, C):
    def hello(self):
        print("D")
        super().hello()

D().hello()
print([c.__name__ for c in D.__mro__])
```

Output:

```
D
B
C
A
['D', 'B', 'C', 'A', 'object']
```

6. Single & Multilevel Inheritance

Q31. Single Inheritance

BEGINNER

Class Phone has call(). Class SmartPhone(Phone) adds browse(). Use both.

Solution:

```
class Phone:
    def call(self):
        print("Calling...")

class SmartPhone(Phone):
    def browse(self):
        print("Browsing internet")

s = SmartPhone()
s.call()
s.browse()
```

Output:

```
Calling...
Browsing internet
```

Q32. Multilevel Inheritance

BEGINNER

Grandparent Person -> Parent Employee -> Child Developer. Each level has one method. Call all three from a Developer object.

Solution:

```
class Person:
    def breathe(self):
        print("Breathing")

class Employee(Person):
    def work(self):
        print("Working")

class Developer(Employee):
    def code(self):
        print("Coding")

d = Developer()
d.breathe()
d.work()
d.code()
```

Output:

```
Breathing
Working
Coding
```

Q33. Constructors in Multilevel

INTERMEDIATE

Person(name) -> Employee(name, emp_id) -> Developer(name, emp_id, language). Use super() at each level and print all values.

Solution:

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

class Developer(Employee):
    def __init__(self, name, emp_id, language):
        super().__init__(name, emp_id)
        self.language = language

d = Developer("Om", "E7", "Python")
print(d.name, d.emp_id, d.language)
```

Output:

```
Om E7 Python
```

Q34. Override at Middle Level

INTERMEDIATE

Person.role() returns "Person", Employee overrides it to "Employee", Developer does not override. What does Developer().role() return?

Solution:

```
class Person:
    def role(self):
        return "Person"

class Employee(Person):
    def role(self):
        return "Employee"

class Developer(Employee):
    pass

print(Developer().role())
```

Output:

```
Employee
```

Q35. Account Hierarchy

ADVANCED

Account(balance) -> SavingsAccount adds interest(rate) -> SeniorSavings overrides interest to add extra 0.5%. Print interest on 100000 at 6%.

Solution:

```
class Account:
    def __init__(self, balance):
        self.balance = balance

class SavingsAccount(Account):
    def interest(self, rate):
        return self.balance * rate / 100

class SeniorSavings(SavingsAccount):
    def interest(self, rate):
        return super().interest(rate + 0.5)

print(SavingsAccount(100000).interest(6))
print(SeniorSavings(100000).interest(6))
```

Output:

```
6000.0
6500.0
```

Q36. Count Levels with MRO

ADVANCED

For the 3-level Developer class, print the full MRO and the number of user-defined classes in it.

Solution:

```
class Person:
    pass

class Employee(Person):
    pass

class Developer(Employee):
    pass

names = [c.__name__ for c in Developer.__mro__]
print(names)
print(len([n for n in names if n != "object"]))
```

Output:

```
['Developer', 'Employee', 'Person', 'object']
3
```

7. Hierarchical, Multiple & Hybrid Inheritance

Q37. Hierarchical Inheritance

BEGINNER

Parent Shape has describe(). Children Square and Triangle both inherit from Shape. Call describe() on both.

Solution:

```
class Shape:
    def describe(self):
        print("I am a", type(self).__name__)

class Square(Shape):
    pass

class Triangle(Shape):
    pass

Square().describe()
Triangle().describe()
```

Output:

```
I am a Square
I am a Triangle
```

Q38. Multiple Inheritance

BEGINNER

Class Camera has click(), class MusicPlayer has play(). Class Smartphone(Camera, MusicPlayer) can do both.

Solution:

```
class Camera:
    def click(self):
        print("Photo clicked")

class MusicPlayer:
    def play(self):
        print("Playing music")

class Smartphone(Camera, MusicPlayer):
    pass

s = Smartphone()
s.click()
s.play()
```

Output:

```
Photo clicked
Playing music
```

Q39. Same Method in Two Parents

INTERMEDIATE

Both Father and Mother have skill(). Child(Father, Mother) calls skill(). Which one runs? Then try Child2(Mother, Father).

Solution:

```
class Father:
    def skill(self):
        return "Driving"

class Mother:
    def skill(self):
        return "Cooking"

class Child(Father, Mother):
    pass

class Child2(Mother, Father):
    pass

print(Child().skill())
print(Child2().skill())
```

Output:

```
Driving
Cooking
```

Q40. Hierarchical with Constructors

INTERMEDIATE

Employee(name) is parent of Teacher(name, subject) and Clerk(name, desk). Create one of each and print details.

Solution:

```
class Employee:
    def __init__(self, name):
        self.name = name

class Teacher(Employee):
    def __init__(self, name, subject):
        super().__init__(name)
        self.subject = subject

class Clerk(Employee):
    def __init__(self, name, desk):
        super().__init__(name)
        self.desk = desk

t = Teacher("Mrs. Joshi", "Maths")
c = Clerk("Mr. Kale", 4)
print(t.name, t.subject)
print(c.name, c.desk)
```

Output:

```
Mrs. Joshi Maths
Mr. Kale 4
```

Q41. Hybrid Inheritance

ADVANCED

School -> (Student, Teacher) and Teaching Assistant(Student, Teacher). Each class has info(). Show the MRO of TA.

Solution:

```
class School:
    def info(self):
        return "School member"

class Student(School):
    def info(self):
        return "Student"

class Teacher(School):
    def info(self):
        return "Teacher"

class TA(Student, Teacher):
    pass

print(TA().info())
print([c.__name__ for c in TA.__mro__])
```

Output:

```
Student
['TA', 'Student', 'Teacher', 'School', 'object']
```

Q42. Mixin Classes

ADVANCED

Create JsonMixin with to_json() using json.dumps(self.__dict__) and a class Product(JsonMixin) with name and price. Convert a product to JSON.

Solution:

```
import json

class JsonMixin:
    def to_json(self):
        return json.dumps(self.__dict__)

class Product(JsonMixin):
    def __init__(self, name, price):
        self.name = name
        self.price = price

print(Product("Mouse", 499).to_json())
```

Output:

```
{"name": "Mouse", "price": 499}
```

8. Method Overloading (Polymorphism)

Q43. Only the Last Definition Survives

BEGINNER

Define add(a, b) and then add(a, b, c) in the same class. Call add with 2 arguments inside try-except to see Python has no real overloading.

Solution:

```
class Calc:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c

c = Calc()
print(c.add(1, 2, 3))
try:
    print(c.add(1, 2))
except TypeError as e:
    print(e)
```

Output:

```
6
Calc.add() missing 1 required positional argument: 'c'
```

Q44. Overloading with Default Values

BEGINNER

Write area(a, b=None): if b is None return square area, else rectangle area.

Solution:

```
class Geometry:
    def area(self, a, b=None):
        if b is None:
            return a * a
        return a * b

g = Geometry()
print(g.area(5))
print(g.area(5, 3))
```

Output:

```
25
15
```

Q45. Overloading with *args

INTERMEDIATE

Write total(*nums) that works with any number of arguments.

Solution:

```
class Calc:
    def total(self, *nums):
        return sum(nums)

c = Calc()
print(c.total(5))
print(c.total(5, 10))
print(c.total(5, 10, 15, 20))
```

Output:

```
5
15
50
```

Q46. Different Behaviour by Type

INTERMEDIATE

Write show(value) that prints "Number" for int/float, "Text" for str and "List of n items" for lists.

Solution:

```
class Printer:
    def show(self, value):
        if isinstance(value, (int, float)):
            print("Number:", value)
        elif isinstance(value, str):
            print("Text:", value)
        elif isinstance(value, list):
            print("List of", len(value), "items")

p = Printer()
p.show(42)
p.show("Pune")
p.show([1, 2, 3])
```

Output:

```
Number: 42
Text: Pune
List of 3 items
```

Q47. Booking with Optional Arguments

ADVANCED

Write book(name, seat=None, meal=None) that builds a message depending on which optional values are given.

Solution:

```
class Booking:
    def book(self, name, seat=None, meal=None):
        msg = "Booked for " + name
        if seat:
            msg += ", seat " + seat
        if meal:
            msg += ", meal " + meal
        return msg

b = Booking()
print(b.book("Om"))
print(b.book("Pia", seat="12A"))
print(b.book("Raj", seat="3C", meal="Veg"))
```

Output:

```
Booked for Om
Booked for Pia, seat 12A
Booked for Raj, seat 3C, meal Veg
```

Q48. singledispatchmethod

ADVANCED

Use functools.singledispatchmethod to create true type-based overloading of process() for int and str.

Solution:

```
from functools import singledispatchmethod

class Processor:
    @singledispatchmethod
    def process(self, value):
        return "Unknown type"

    @process.register
    def _(self, value: int):
        return value * 2

    @process.register
    def _(self, value: str):
        return value.upper()

p = Processor()
print(p.process(21))
print(p.process("pune"))
print(p.process(3.5))
```

Output:

```
42
PUNE
Unknown type
```

9. Operator Overloading (Polymorphism)

Q49. Same Operator, Different Types

BEGINNER

Show that + adds numbers, joins strings and joins lists (built-in operator overloading).

Solution:

```
print(10 + 20)
print("Bikkad" + " IT")
print([1, 2] + [3, 4])
```

Output:

```
30
Bikkad IT
[1, 2, 3, 4]
```

Q50. Add Two Money Objects

BEGINNER

Create class Money(amount) with __add__ and __str__. Add Money(250) + Money(400).

Solution:

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __str__(self):
        return "Rs." + str(self.amount)

print(Money(250) + Money(400))
```

Output:

```
Rs.650
```

Q51. Compare Distances

INTERMEDIATE

Create class Distance(km) with __gt__ and __eq__. Compare Distance(12) > Distance(8) and Distance(5) == Distance(5).

Solution:

```
class Distance:
    def __init__(self, km):
        self.km = km

    def __gt__(self, other):
        return self.km > other.km

    def __eq__(self, other):
        return self.km == other.km

print(Distance(12) > Distance(8))
print(Distance(5) == Distance(5))
```

Output:

```
True
True
```

Q52. Multiply a Vector

INTERMEDIATE

Create class Vector(x, y) with __mul__ for multiplying by a number and __str__.

Solution:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __mul__(self, n):
        return Vector(self.x * n, self.y * n)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

print(Vector(2, 3) * 4)
```

Output:

```
Vector(8, 12)
```

Q53. Fraction Class

ADVANCED

Create class Fraction(num, den) supporting + and == and printing like 3/4. Add 1/2 + 1/4 (reduce using math.gcd).

Solution:

```
import math

class Fraction:
    def __init__(self, num, den):
        g = math.gcd(num, den)
        self.num = num // g
        self.den = den // g

    def __add__(self, other):
        n = self.num * other.den + other.num * self.den
        d = self.den * other.den
        return Fraction(n, d)

    def __eq__(self, other):
        return self.num == other.num and self.den == other.den

    def __str__(self):
        return f"{self.num}/{self.den}"

print(Fraction(1, 2) + Fraction(1, 4))
print(Fraction(2, 4) == Fraction(1, 2))
```

Output:

```
3/4
True
```

Q54. Cart Total with sum()

ADVANCED

Make sum() work on a list of Money objects by adding __radd__ (sum starts with 0).

Solution:

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)

    def __radd__(self, other):
        if other == 0:
            return self
        return self.__add__(other)

    def __str__(self):
        return "Rs." + str(self.amount)

cart = [Money(120), Money(450), Money(80)]
print(sum(cart))
```

Output:

```
Rs.650
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026